JAVA 2

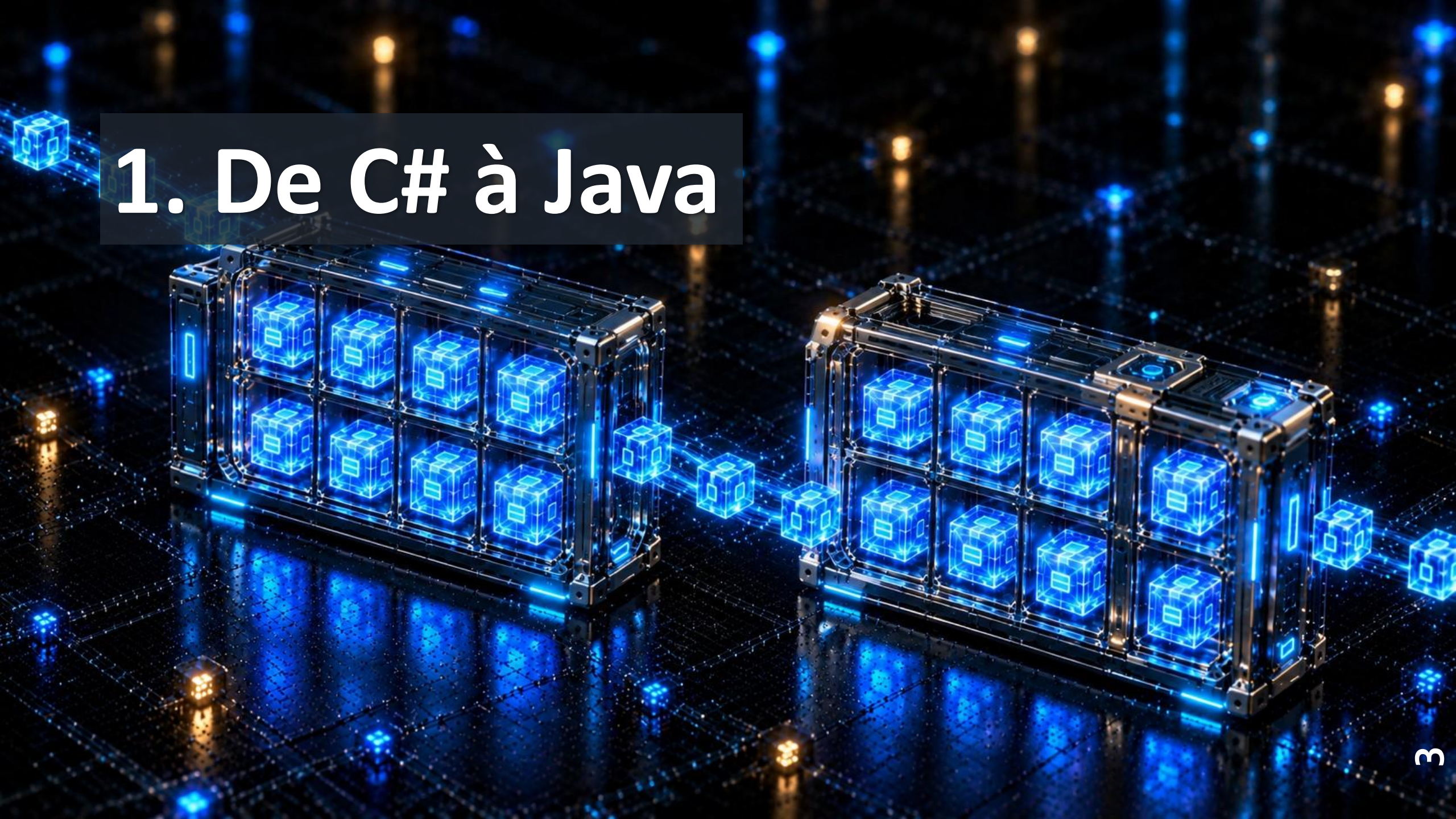
Module POO

Classes et objets

Au programme

1. De C# à Java
2. Classe et objet
3. Encapsulation
4. Membres de classe (`static`)
5. Saisie clavier et Javadoc
6. Primitifs et références
7. Les wrappers

1. De C# à Java



Pas de grosses différences niveau langage

C#

```
namespace ProgOO
{
    class Personne
    {
        public string Nom;
        public string Prenom;
         public int AnneeNaissance;
         public int Age;
        public void CalculAge(int Annee)
        {
            Age = Annee - AnneeNaissance;
        }
         public void Afficher()
        {
            Console.WriteLine(Prenom + " " +
              Nom + " (" + Age + " ans)");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
              [...]
        }
    }
}
```

java

```
package progoo;

class Personne {
    public String nom;
    public String prenom;
     public int anneeNaissance;
     public int age;

     public void calculAge(int annee) {
          age = annee - anneeNaissance;
    }

     public void afficher() {
          System.out.println(prenom + " " +
               nom + " (" + age + " ans)");
     }
}

public class Program {
     public static void main(String[] args) {
          [...]
    }
}
```

Quelles différences repérez-vous ?

Structuration des classes

C#

```
namespace ProgOO
{
    class Personne
    {
        public string Nom;
        public string Prenom;
         public int AnneeNaissance;
         public int Age;
        public void CalculAge(int Annee)
        {
            Age = Annee - AnneeNaissance;
        }
         public void Afficher()
        {
            Console.WriteLine(Prenom + " " +
              Nom + " (" + Age + " ans)");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
             [...]
        }
    }
}
```

package pour structurer les classes

```
package progoo;

class Personne {
    public String nom;
    public String prenom;
     public int anneeNaissance;
     public int age;

     public void calculAge(int annee) {
          age = annee - anneeNaissance;
     }

     public void afficher() {
          System.out.println(prenom + " " +
               nom + " (" + age + " ans)");
     }
}

public class Program {
     public static void main(String[] args) {
          [...]
     }
}
```

java

Conventions de nommage différentes

C#

```
namespace ProgOO
{
    class Personne
    {
        public string Nom;
        public string Prenom;
         public int AnneeNaissance;
         public int Age;
        public void CalculAge(int Annee)
        {
            Age = Annee - AnneeNaissance;
        }
         public void Afficher()
        {
            Console.WriteLine(Prenom + " " +
              Nom + " (" + Age + " ans)");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
             [...]
        }
    }
}
```

java

```
package progoo;

class Personne {
    public String nom;
    public String prenom;
     public int anneeNaissance;
     public int age;

     public void calculAge(int annee) {
          age = annee - anneeNaissance;
     }

     public void afficher() {
          System.out.println(prenom + " " +
               nom + " (" + age + " ans)");
     }
}

public class Program {
     public static void main(String[] args) {
          [...]
     }
}
```

Majuscule pour les noms de classes, enumérations et interfaces.

minuscule pour les variables, méthodes et packages

Lien entre fichier et classe

C#

```
namespace ProgOO
{
    class Personne
    {
        public string Nom;
        public string Prenom;
         public int AnneeNaissance;
         public int Age;
        public void CalculAge(int Annee)
        {
            Age = Annee - AnneeNaissance;
        }
         public void Afficher()
        {
            Console.WriteLine(Prenom + " " +
              Nom + " (" + Age + " ans)");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
             [...]
        }
    }
}
```

java

```
package progoo;

class Personne {
    public String nom;
    public String prenom;
     public int anneeNaissance;
     public int age;

     public void calculAge(int annee) {
          age = annee - anneeNaissance;
     }

     public void afficher() {
          System.out.println(prenom + " " +
               nom + " (" + age + " ans)");
     }
}

public class Program {
     public static void main(String[] args) {
          [...]
     }
}
```

Dans un fichier, une seule classe `public`

Classes du Framework différentes

C#

```
namespace ProgOO
{
    class Personne
    {
        public string Nom;
        public string Prenom;
         public int AnneeNaissance;
         public int Age;
        public void CalculAge(int Annee)
        {
            Age = Annee - AnneeNaissance;
        }
         public void Afficher()
        {
            Console.WriteLine(Prenom + " " +
              Nom + " (" + Age + " ans)");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
             [...]
        }
    }
}
```

```
package progoo;

class Personne {
    public String nom;
    public String prenom;
     public int anneeNaissance;
     public int age;

     public void calculAge(int annee) {
          age = annee - anneeNaissance;
     }

     public void afficher() {
          System.out.println(prenom + " " +
               nom + " (" + age + " ans)");
     }
}

public class Program {
     public static void main(String[] args) {
          [...]
     }
}
```

java

`System.out.println` : les classes du framework sont différentes

Méthode main rend la classe exécutable

C#

```
namespace ProgOO
{
    class Personne
    {
        public string Nom;
        public string Prenom;
         public int AnneeNaissance;
         public int Age;
        public void CalculAge(int Annee)
        {
            Age = Annee - AnneeNaissance;
        }
         public void Afficher()
        {
            Console.WriteLine(Prenom + " " +
              Nom + " (" + Age + " ans)");
        }
    }
    class Program
    {
        static void Main(string[] args)
        {
            [...]
        }
    }
}
```

```
package progoo;

class Personne {
    public String nom;
    public String prenom;
     public int anneeNaissance;
     public int age;

     public void calculAge(int annee) {
          age = annee - anneeNaissance;
     }

     public void afficher() {
          System.out.println(prenom + " " +
               nom + " (" + age + " ans)");
     }
}

public class Program {
     public static void main(String[] args) {
          [...]
     }
}
```

La méthode `main` doit être `public`

java

Classe Java et Programme (exécutable) Java

En java, n'importe quelle classe peut être exécutable.

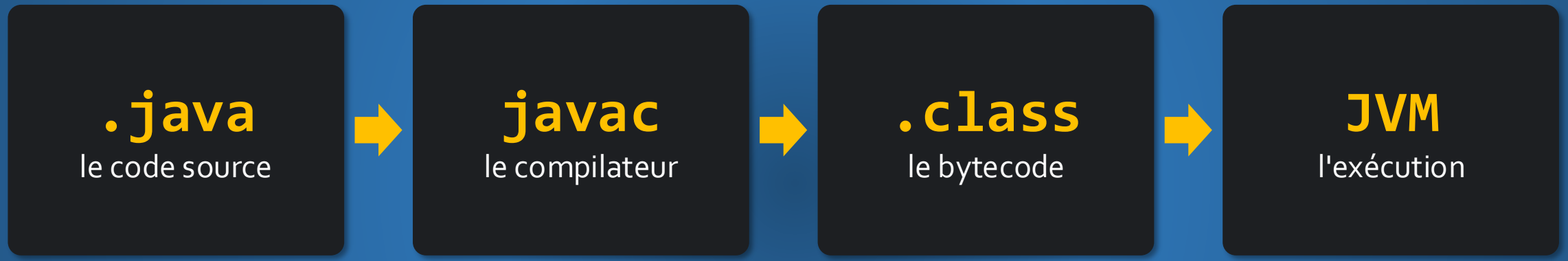
Il suffit qu'elle contienne une méthode qui a la signature suivante:

```
public static void main(String[] args)
```

Pour tester un ensemble de classes, je vous conseille plutôt d'écrire une classe `Programme` qui utilisera les classes en question.

Nous verrons plus tard une autre façon de tester si une classe fonctionne correctement.

Du code source au programme



Le détail : mémento technique, **TEC-CMP.1** et **TEC-CMP.2**

En résumé…

`namespace` devient `package`

Majuscule pour les types, minuscule pour le reste

Un fichier `.java` = au plus une classe `public`, qui lui donne son nom

`Console.WriteLine` devient `System.out.println`

`main` doit être `public static`

2. Classe et objet



Les quatre principes de la POO

Encapsulation

L'objet protège son état

→ section 3

Abstraction

Ne modéliser que ce qui compte pour le problème

Héritage

Une classe reprend et étend une autre

→ module ABS

Polymorphisme

Traiter uniformément des objets différents

→ modules ABS et ITF

Classe et objet

Une **classe** est un moule : elle décrit des champs et des méthodes

Un **objet** est un exemplaire construit à partir du moule

`new` crée l'objet

la variable à laquelle l'objet est assigné par `=` permet de retrouver l'objet

```java
public class Livre {
    public String titre;
    public String auteur;
    public int anneeDeParution;
}
```

```java
Livre dune = new Livre();
dune.titre = "Dune";
```

Démonstration



Démonstration

Construire une classe, puis l'encapsuler

Déroulé : Demo_POO

Le constructeur

Il porte le nom de la classe, n'a pas de type de retour

Il est appelé par new, il initialise l'objet

Sans constructeur écrit, Java en fournit un « par défaut », sans paramètre

Dès qu'on écrit un constructeur « explicite », le constructeur par défaut disparaît

```java
public class Livre {
    private String titre;
    private int anneeDeParution;

    public Livre(String titre, int anneeDeParution) {
        this.titre = titre;
        this.anneeDeParution = anneeDeParution;
    }
}
```

```java
Livre dune = new Livre("Dune", 1965);
// refuse : ce constructeur n'existe plus
Livre vide = new Livre();
```

Le mot-clé `this`

`this` désigne l'objet en cours

Il lève l'ambiguïté quand un paramètre porte le nom d'un champ

Piège : `titre = titre;` compile, mais ne fait rien

```
public Livre(String titre, int anneeDeParution) {
    this.titre = titre;   // le champ recoit le parametre
    titre = titre;        // le parametre recoit... lui-meme
}
```

Un champ `final`

`final` : le champ reçoit sa valeur une seule fois, au plus tard dans le constructeur

Le compilateur refuse toute affectation ultérieure

L'immuabilité n'est plus une affaire de vigilance mais est vérifiée par le compilateur

```
public class Livre {
    private final String titre;

    public Livre(String titre) {
        this.titre = titre;
    }

    public void renommer(String nouveauTitre) {
        titre = nouveauTitre;
        // refuse : cannot assign a value
        //          to final variable titre
    }
}
```

La méthode `toString`

Tout objet sait se transformer en texte, grâce à sa méthode `toString` (présente dans `Object`)

La méthode retourne par défaut : nom de la classe + `@` + un numéro

On redéfinit la méthode `toString` pour afficher quelque chose d'utile

Cette méthode est appelée implicitement par `println` et par la concaténation

```java
@Override
public String toString() {
    return titre + " (" + anneeDeParution + ")";
}

System.out.println(dune);          // Dune (1965)
String texte = "Lu : " + dune;     // Lu : Dune (1965)
```

Exercices

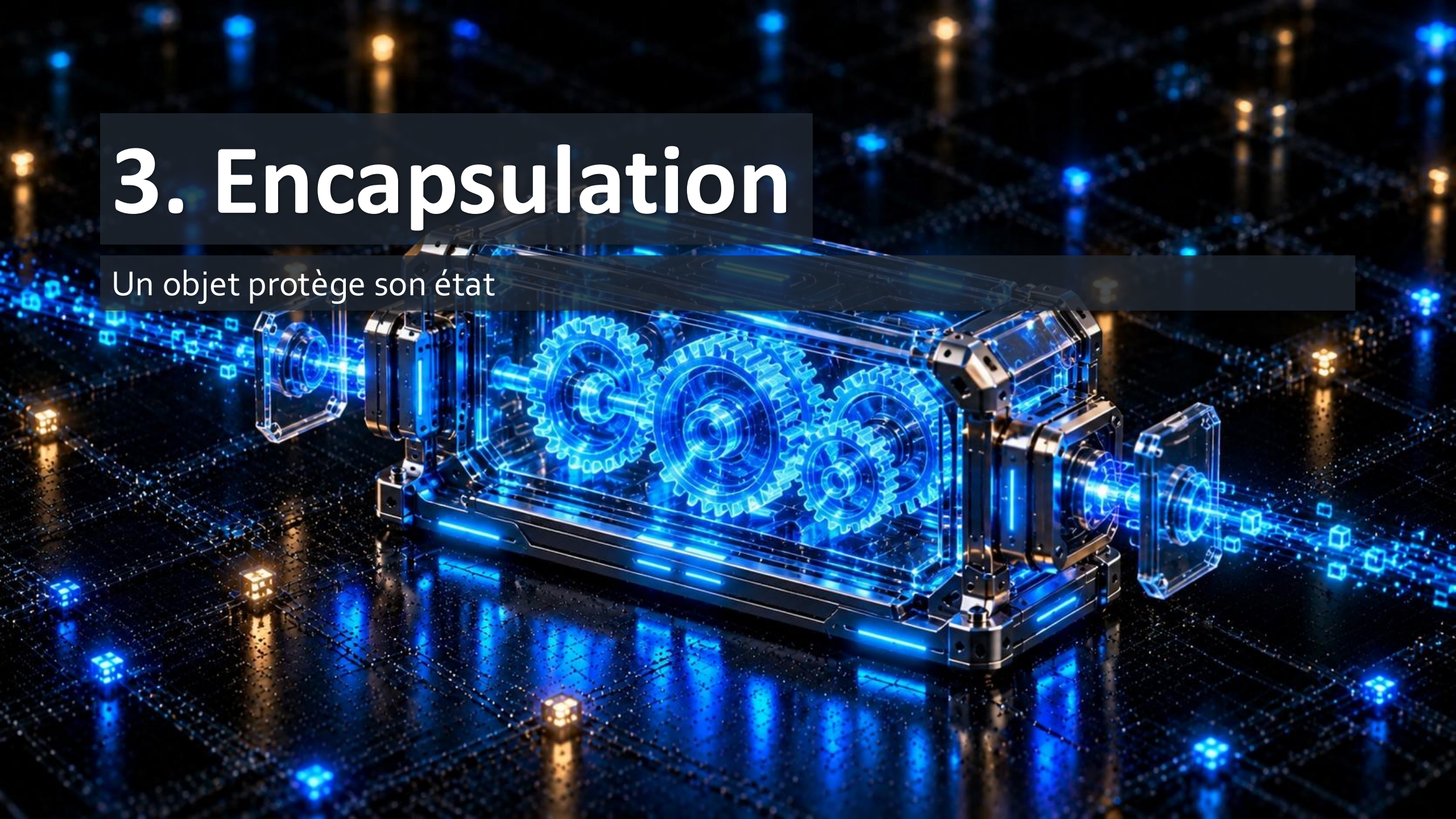
POO.1	Une première classe fascicule POO
G.1	Le joueur fil rouge GameHub
G.2	La partie fil rouge GameHub



Nommer : `CC-NOM.1` et `CC-NOM.5`

3. Encapsulation

Un objet protège son état



Le problème d'un champ `public`

Tout le monde peut écrire n'importe quoi

Le programme est faux, et rien ne le signale

```
Livre dune = new Livre("Dune", 1965);
dune.anneeDeParution = -3000;   // champ public : accepte sans broncher
```

Fermer les champs, ouvrir des accesseurs

Les champs deviennent `private`

Getter : `getTitre()` lit la valeur

Setter : `setTitre(String)` la modifie

Pour un booléen, le getter commence par `is` : `isDisponible()`

```
public class Livre {
    private String titre;

    public String getTitre() {
        return titre;
    }

    public void setTitre(String titre) {
        this.titre = titre;
    }
}
```

Ce que le détour permet

Le setter est l'endroit où l'on place une règle

Un champ `public` ne le permettra jamais

`IllegalArgumentException` : non contrôlée, pas de `try/catch` à écrire (module EXC)

```
public void setAnneeDeParution(int annee) {
    if (annee < 0) {
        throw new IllegalArgumentException("Annee invalide : " + annee);
    }
    this.anneeDeParution = annee;
}
```

Un accesseur n'est pas une obligation

Attention, générer systématiquement un getter et un setter par champ annule l'encapsulation

Chaque accesseur doit avoir une raison d'être

Un champ `final` n'a pas de setter

Voir `CC-OBJ.2`

La donnée qui n'existe pas

Un champ age se périme tout seul

Ce qu'on peut calculer n'a pas à être stocké

Une méthode (de type getter) remplace le champ

```
public int getAge() {
    return LocalDate.now().getYear() - anneeDeParution;
}
```

Exercices



POO.2 La même classe, encapsulée

fascicule POO

POO.3 L'âge n'est pas une donnée à stocker

fascicule POO

Générer les accesseurs : IDE.12



4. static

`static` = défini au niveau de la classe, pas pour chaque objet

Champs et méthodes de classe

Il est possible, en Java, de définir des champs qui, plutôt que d'exister dans chaque instance de la classe, existent en un seul exemplaire pour toutes les instances de la classe.

On les appelle les champs de classe, ou champs statiques.

On peut aussi définir des méthodes de classe (statiques), qui peuvent être appelées indépendamment de tout objet (donc sans nécessairement devoir créer d'objet).

C'est le cas de la méthode `main()`, mais aussi des différentes méthodes rencontrées sur la classe `Math` (`Math.sqrt()`, par exemple).

Champs d'objets

Premier exemple

1. Soit une classe très simple:

```
class A { int n;  float y; }
```

Chaque objet de type A possède ses propres champs n et y.

```
A a1 = new A();    A a2 = new A();
```



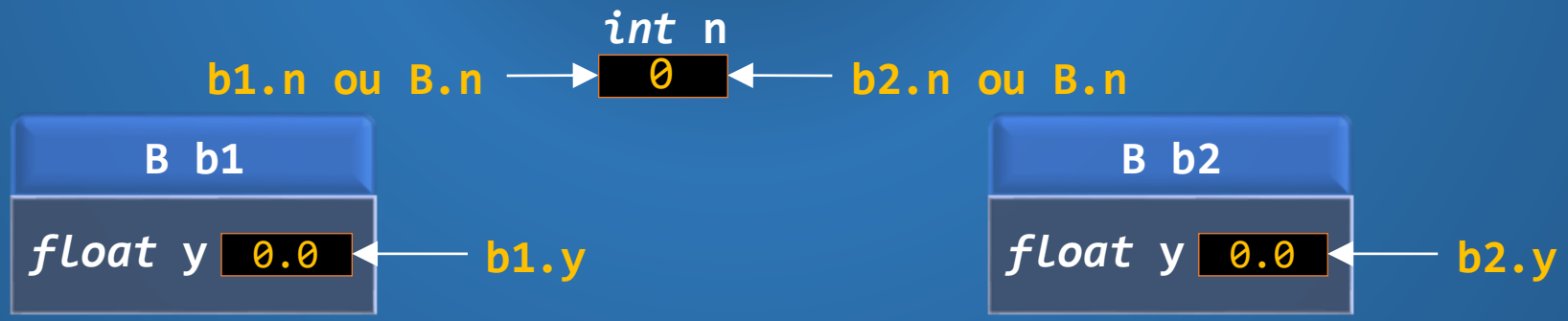
Champ de classe

Définissons une autre classe dans laquelle le premier champ est `static`:

```
class B { static int n;  float y;  }
```

Chaque objet de type `B` possède son propre champ `y`, mais `n` est maintenant commun (défini une seule fois pour toutes les instances de la classe).

```
B b1 = new B();   B b2 = new B();
```



Nommer un champ de classe

Une fois que le champ est `static`, les notations

`b1.n` `b2.n`

définissent le même champ, qui existe en fait pour toutes les instances de la classe.

Il est dès lors préférable de le nommer

```
B.n  // le champ static n de la classe B
```

Ces notations ne sont valables que si `n` est non privé.

On peut cependant définir des champs `static` privés, qui sont alors accessibles via des méthodes de la classe.

Et… depuis une méthode de `B`, le champ peut être nommé `n` (pas besoin d'écrire `B.n`)

Démonstration



Démonstration

Ce qui appartient à la classe, ce qui appartient à l'objet

Déroulé : Demo_STA

Un compteur d'instances

Un champ statique compte les objets créés

Question : qu'affiche ce programme ?

```
class Ticket {
    private static int crees = 0;

    public Ticket() {
        crees++;
        System.out.println("Ticket " + crees + " cree");
    }
}

public class Programme {
    public static void main(String[] args) {
        Ticket a;
        System.out.println("main 1");
        a = new Ticket();
        System.out.println("main 2");
        Ticket b;
        System.out.println("main 3");
        b = new Ticket();
        Ticket c = new Ticket();
        System.out.println("main 4");
    }
}
```

Méthodes de classe

Elles ne dépendent d'aucun objet

Elles s'appellent sur la classe : `Math.sqrt(16)`

Il est possible d'appeler une méthode de classe sur un objet, mais c'est peu lisible et trompeur : à éviter

```
double racine = Math.sqrt(16);
int annee = LocalDate.now().getYear();
```

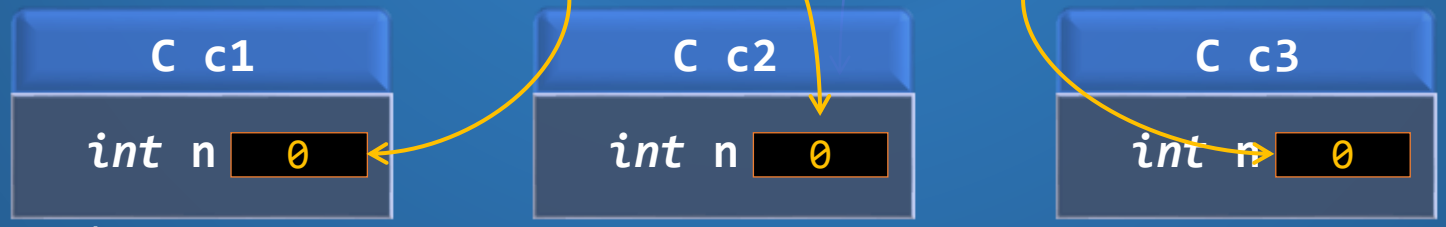
Une méthode statique n'a pas de `this`

Une méthode statique ne peut pas faire référence à des variables ni des méthodes qui ne sont pas statiques…

En effet, essayons d'accéder à n, champ non statique, depuis une méthode statique, donc définie au niveau de la classe…

```
class C {
  int n = 0;
  public static void fixer() { n = 2; }
}
```

`C.fixer();` → **n, oui, mais lequel?**



Les constantes

`static` : une seule valeur, partagée

`final` : elle ne change pas

Une constante a besoin des deux : `static` et `final`

Convention de nommage: nom en majuscules, mots séparés par _

```
public class Facture {
    public static final double TVA = 0.21;
    public static final int JOURS_DE_PAIEMENT = 30;
}

double prix = 100 * (1 + Facture.TVA);
```

Quatre sortes de champs

	Peut changer	Ne change plus : `final`
Propre à chaque objet	`private int anneeDeParution;` l'année d'un livre, qu'un setter peut corriger	`private final String titre;` le titre, initialisé une fois pour toutes par le constructeur
Partagé par la classe : `static`	`private static int crees;` un compteur d'instances	`public static final double TVA = 0.21;` une constante

À quoi servent les méthodes de classe

Répondre sur l'ensemble : combien d'objets ont été créés ?

Offrir un service sans état : `Math.sqrt`, `Integer.parseInt`

Fabriquer des objets : `LocalDate.now()`, `LocalDate.of(...)`

Les fabriques

Un constructeur porte le nom de sa classe, et rien d'autre

Une fabrique porte le nom qu'on veut

Deux fabriques peuvent avoir la même signature

Quand on crée des fabriques, le constructeur est `private` : on passe obligatoirement par les fabriques pour créer des objets

```java
public class Point {
    private final double x;
    private final double y;

    private Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public static Point cartesien(double x, double y) {
        return new Point(x, y);
    }

    public static Point polaire(double rayon, double angle) {
        double x = rayon * Math.cos(angle);
        double y = rayon * Math.sin(angle);
        return new Point(x, y);
    }
}

Point a = Point.cartesien(3, 4);
Point b = Point.polaire(5, 0.93);
```

Exercices



STA.1 Une constante et un compteur d'instances
fascicule STA

STA.2 Un constructeur privé et des fabriques
fascicule STA

Le débogueur : IDE.19

5. Entrée de données au clavier



Pour les entrées au clavier, on utilise la classe `Scanner`…

Cette classe nous permet de récupérer ce qui a été entré par l'utilisateur à la console. `Scanner` a plusieurs méthodes utiles: `nextLine()`, `nextInt()`,….

```java
import java.util.*;

public class InputTest {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        System.out.print("Comment vous appelez-vous? ");
        String name = in.nextLine();

        System.out.print("Quel age avez-vous? ");
        int age = in.nextInt();

        System.out.println("Bonjour " + name + ". Dans 1 an, vous aurez " + (age + 1) + " ans.");
        in.close();
    }
}
```

La Javadoc

Pour connaître les caractéristiques d'une classe Java, il suffit de consulter la Javadoc:
https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html

Dans IntelliJ : survol du nom, ou `Ctrl+Q` / `F1` - voir `IDE.14`

Une page Javadoc, au format HTML, reprend toujours

Une explication générale de la classe, avec parfois des détails sur son utilisation

Un inventaire des constructeurs de la classe, et un inventaire des méthodes de la classe.

Une description de chaque constructeur et de chaque méthode, décrivant chaque fois

- les paramètres,
- les valeurs retournées
- les éventuelles Exceptions.

Pour notre classe `Scanner`, de quelles méthodes pourrions-nous avoir besoin (quelles valeurs lire à la console)?

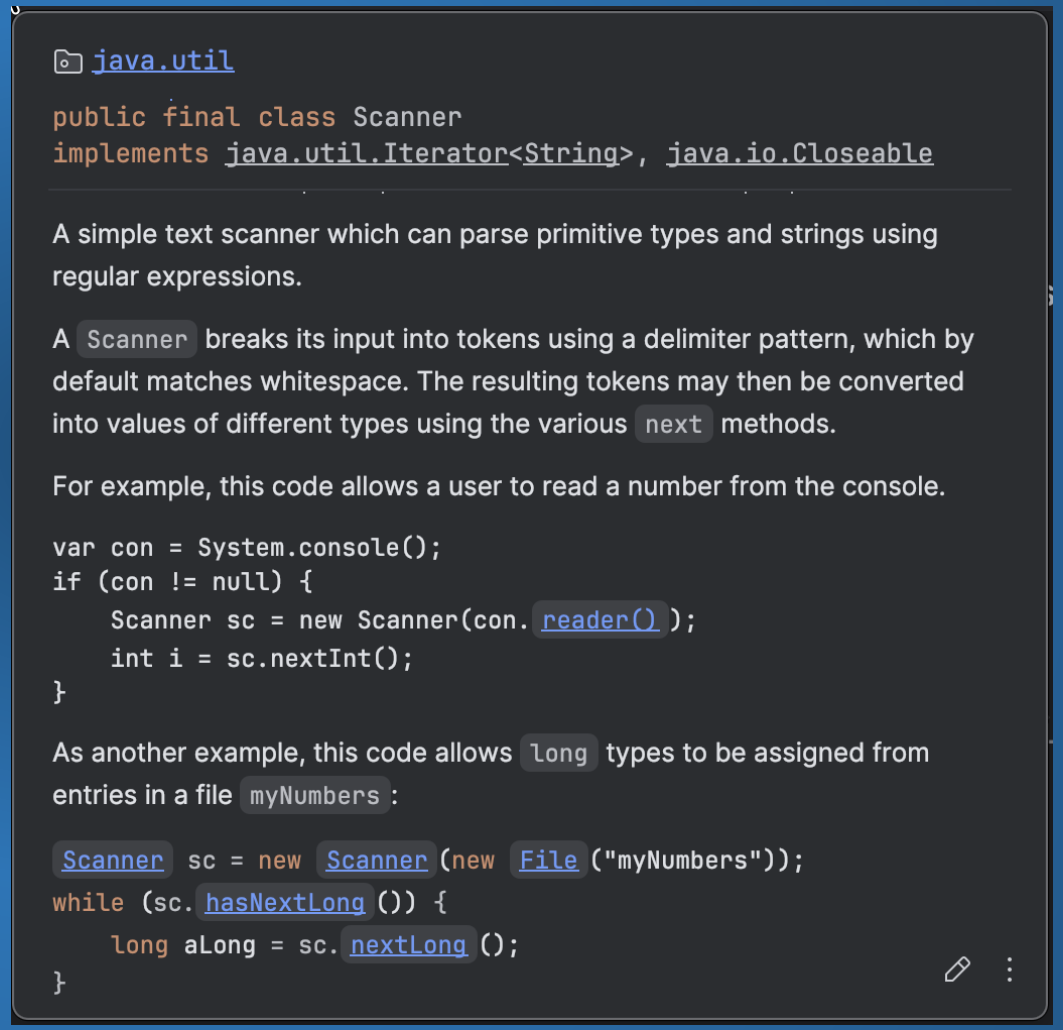
La documentation de Scanner

Survol de `Scanner` dans IntelliJ, ou `Ctrl+Q` / `F1` (`IDE.14`)

On voit que `Scanner` est une classe du package `java.util`

On voit aussi qu'elle découpe l'entrée en jetons, séparés par des blancs

La classe présente une méthode `next...` par type à lire



java.util

```
public final class Scanner
implements java.util.Iterator<String>, java.io.Closeable
```

A simple text scanner which can parse primitive types and strings using regular expressions.

A `Scanner` breaks its input into tokens using a delimiter pattern, which by default matches whitespace. The resulting tokens may then be converted into values of different types using the various `next` methods.

For example, this code allows a user to read a number from the console.

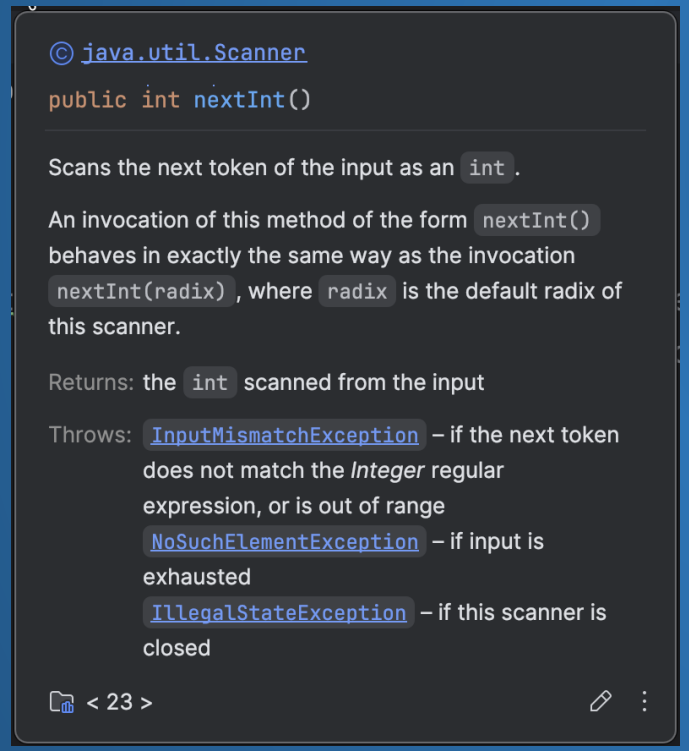
```
var con = System.console();
if (con != null) {
    Scanner sc = new Scanner(con.reader());
    int i = sc.nextInt();
}
```

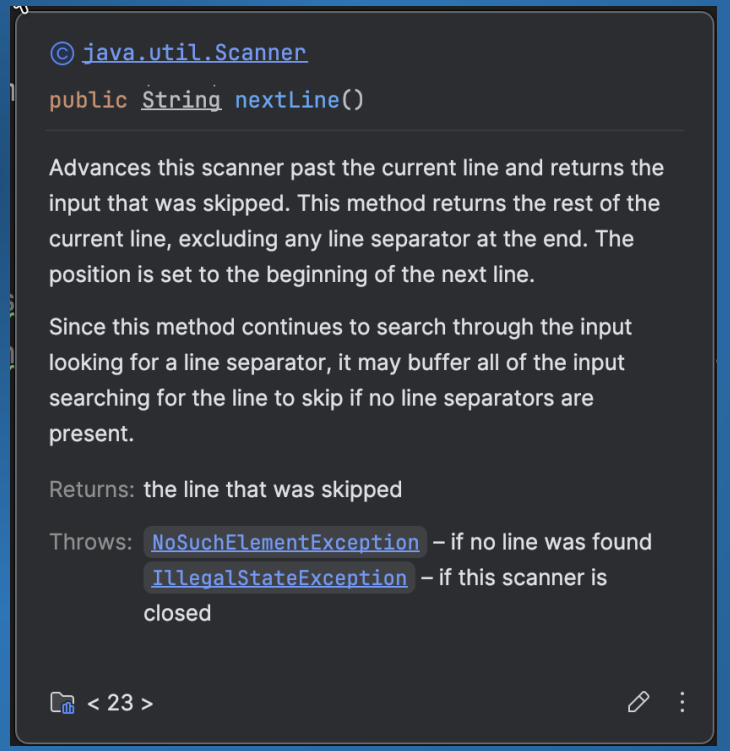
As another example, this code allows `long` types to be assigned from entries in a file `myNumbers`:

```
Scanner sc = new Scanner(new File("myNumbers"));
while (sc.hasNextLong()) {
    long aLong = sc.nextLong();
}
```

Les principales méthodes de `Scanner`

Note: les méthodes `nextInt`, `nextByte`,… (pour les valeurs entières) acceptent aussi un `int` pour indiquer la base dans laquelle le nombre doit être interprété.





Un piège : `nextInt` suivi de `nextLine`

`nextInt()` lit le nombre, mais laisse la fin de ligne

Le `nextLine()` suivant la ramasse et retourne une chaîne vide

La question semble sautée

```
System.out.print("Annee : ");
int annee = clavier.nextInt();        // lit 1965, laisse la fin de ligne
System.out.print("Titre : ");
String titre = clavier.nextLine();    // retourne "" sans attendre
```

Exercices

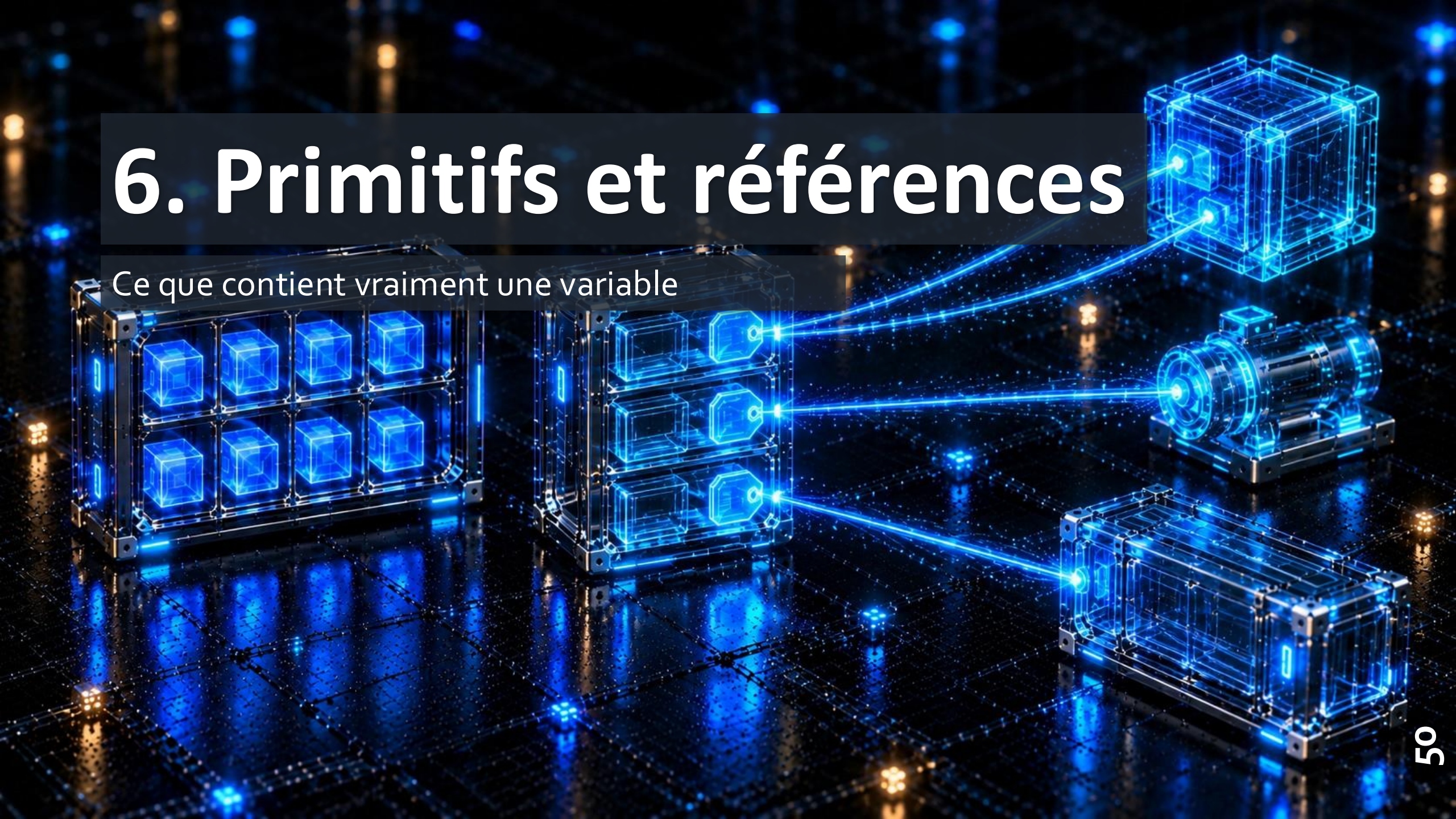
POO.4 Saisie au clavier, **première partie seulement**
fascicule POO

G.3 Le point d'entrée
fil rouge GameHub



6. Primitifs et références

Ce que contient vraiment une variable



Types primitifs vs Objets

Vous connaissez déjà la différence entre les 8 types primitifs (`int`, `boolean`, `double`,...) et les objets, instances de classes.

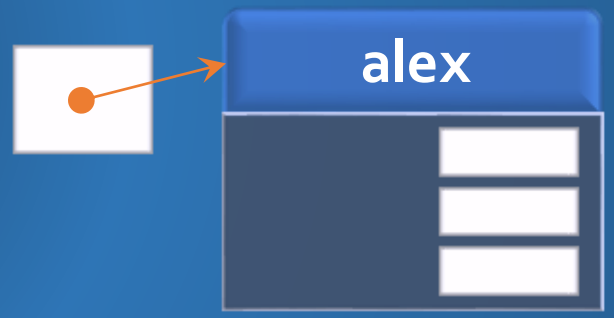
type primitif

```
int hauteur
```

200

objet

```
Etudiant alex
```



Les valeurs des variables de type primitif sont stockées dans les variables, alors que les valeurs contenues dans les variables de type objet sont des références vers les objets.

Deux noms, un seul objet

Copier une variable primitive copie la valeur

Copier une variable d'objet copie la référence

Les deux variables désignent alors le même objet

```
int a = 5;
int b = a;
b = 7;                                  // a vaut toujours 5

Livre l1 = new Livre("Dune", 1965);
Livre l2 = l1;                          // l2 designe le MEME objet
l2.setAnneeDeParution(1966);
System.out.println(l1.getAnneeDeParution()); // 1966
```

Exercice



POO.4 Saisie au clavier, **seconde partie**

fascicule POO

Observez, puis expliquez par écrit, avant de passer à la slide suivante

Ce que vous avez observé

Java passe toujours les arguments **par valeur**

Pour un `int` : une copie du nombre

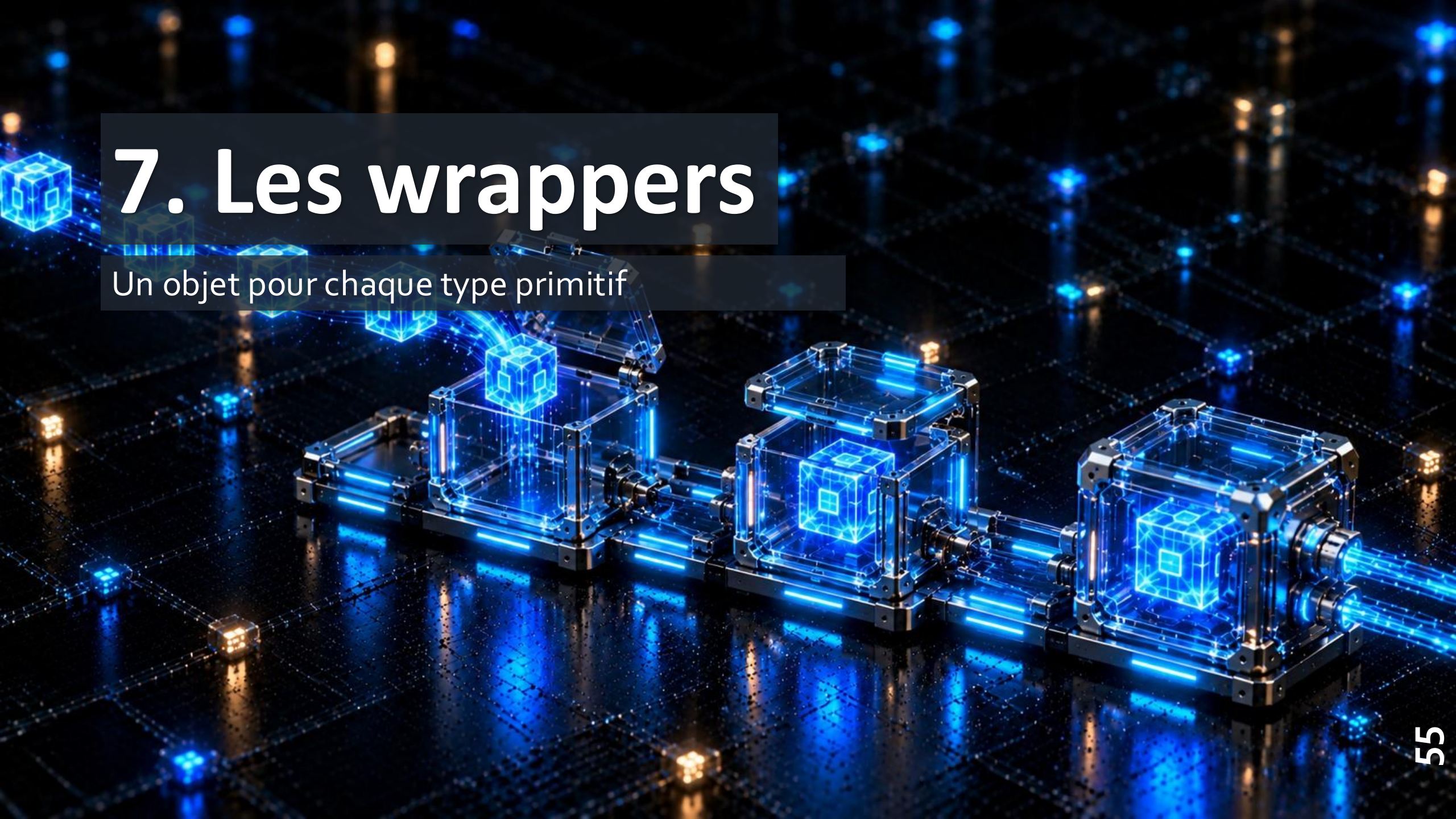
Pour un objet : une copie de la référence, qui désigne le même objet

Réaffecter le paramètre ne change rien chez l'appelant

```
static void modifier(int n, Livre livre) {
    n = 99;                              // change la copie
    livre.setAnneeDeParution(2000);      // change l'objet partage
    livre = new Livre("Autre", 1);       // la copie designe un autre objet
}
```

7. Les wrappers

Un objet pour chaque type primitif



Types primitifs et Wrappers

Pour chaque type primitif (`boolean`, `short`, `byte`, `int`, `long`, `char`, `double`, `float`), il existe une classe correspondante appelée Wrapper.

Type Primitif	Wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Ce qu'offrent les wrappers

Des conversions :
`Integer.parseInt(String)`

Des constantes :
`Integer.MAX_VALUE`,
`Integer.MIN_VALUE`

Des tests :
`Character.isDigit(char)`,
`Character.isWhitespace(char)`

Tous statiques

Autoboxing et unboxing : la conversion est automatique

```
int n = Integer.parseInt("42");
int plusGrand = Integer.MAX_VALUE;
boolean chiffre = Character.isDigit('7');

Integer boite = 5;      // autoboxing
int valeur = boite;     // unboxing
```

Séparer le calcul du dialogue

Une méthode calcule et retourne un résultat : elle ne lit ni n'affiche rien

Le `main` dialogue avec l'utilisateur : il ne calcule rien

Le calcul devient testable

Voir `CC-MET.8`

```java
public class Disque {
    public static double aire(double rayon) {
        return Math.PI * rayon * rayon;
    }

    public static void main(String[] args) {
        Scanner clavier = new Scanner(System.in);
        System.out.print("Rayon : ");
        double rayon = clavier.nextDouble();
        System.out.println("Aire : " + aire(rayon));
    }
}
```

Exercice



POO.5 Changement de base, et logique testable

fascicule POO

La Javadoc de `Integer.parseInt(String, int)` : `IDE.14`

En résumé

Une classe décrit, `new` construit, le constructeur initialise

`this` désigne l'objet en cours ; `final` fige un champ

Des champs `private`, et des accesseurs seulement s'ils se justifient

`static` : ce qui appartient à la classe, pas à l'objet

Une variable d'objet contient une référence

Le calcul dans une méthode, le dialogue dans le `main`